

Each channel can be configured to operate in input capture or output compare mode, as controlled by the mode select in CCyS in TIMx_CCMR1 or TIMx_CCMR2. The fields of these registers are dependent on the mode chosen for the channel and have a different function in input or output mode. The register fields of TIMx_CCMR1 when the channel is configured in output mode are shown in the upper row of Figure 7.22, while those for input mode are shown in the lower row.

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Output	OC2CE	OC2M			OC2PE	OC2FE	CC2S		OC1CE	OC1M			OC1PE	OC1FE	CC1S	
Input	IC2F				IC2PSC		CC2S		IC1F				IC1PSC		CC1S	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Access	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 7.22 TIMx_CCMR1 channel 1 and 2 control register.

The TIMxSR register holds status flags for all channels. Each channel has two flags: the CCyOF flag for overcapture, and CCyIF for interrupt. There is also the UIF flag to indicate a pending update interrupt and the TIF flag to indicate a trigger interrupt is pending. Grouping the flags in one register simplifies the code needed to identify which channel flags are set.

Input Capture Mode

Using input capture mode makes time measurement of input signals much less vulnerable to delays in processing interrupts because the channel hardware automatically captures the timer value without the need for any software intervention.

When the channel is in input capture mode, the input signal is monitored for a specific transition (rising edge, falling edge, or either). When that transition occurs, the value of the TIMx_CNT register is captured in the TIMx_CCRy register, and the channel flag CCyIF in TIMx_SR is set to one. If another consecutive capture happens before CCyIF is cleared, CCyOF is set as well to indicate an overcapture error condition. An interrupt may be generated depending on the CCyIE bit of TIMx_DIER. The ISR must copy the captured value out of the channel's TIMx_CCRy register and use it. The ISR must also clear the channel flag by writing zero to it. If the DMA field CCyDE is set in TIMx_DIER, then a DMA transfer will be requested.

We configure timer's channel as follows:

- Select input capture mode in CCyS in TIMx_CCMR1 or TIMx_CCMR2 by setting the bits to 01, 10 or 11 depending on the trigger event source.
- Optionally enable trigger filtering to remove noise. After an input trigger changes, it must be stable for this amount of time to be recognized. In the TIMx_CCMR1 or TIMx_CCMR2 register set the ICyF field to the sampling value.
- Select which edges trigger a capture using CCyNP:CCyP field in TIMx_CCER: 0:0 for rising edges, 0:1 for falling edges, 1:1 for both rising and falling edges.
- Select the input prescaler. The capture can be done after each valid transition or after 2, 4, or 8 valid transitions by setting the ICyPS bits of TIMx_CCMR1 or TIMx_CCMR2.
- Enable the capture register by setting CCyE bit in the TIMx_CCER register to one.
- If used, enable interrupts by setting CCyIE to one in the TIMx_DIER register.
- If used, enable a DMA request by setting CCyDE to one in the TIMx_DIER register.

Example: Anemometer Using Period Measurement

Let's see how to use the input capture mode to measure the period of the input signal from an anemometer. The circuit is shown in Figure 7.23. The anemometer switch B3 bounces when it closes, generating extra pulses which the hardware and software will count, in turn corrupting the measurement. Capacitor C1 is connected across the switch contacts to filter out switch bounce. Please refer to Figure 7.8 for signal locations.

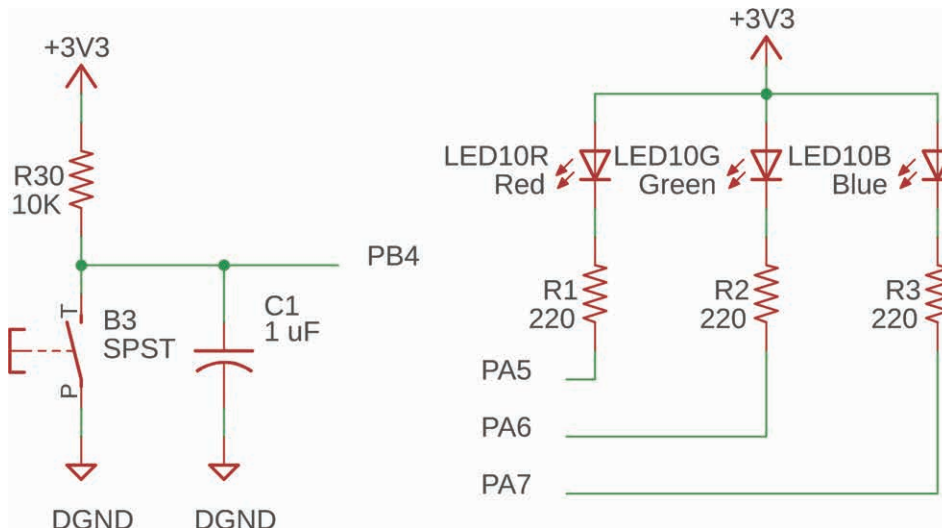


Figure 7.23. Circuit for anemometer with RGB LED output.

This initialization code for the TIM3 is shown in Listing 7.10.

```
#define F_TIM_CLOCK (48UL*1000UL*1000UL) // 48 MHz
#define TIM_PRESCALER (4800)

void Init_TIM_IC(void) {
    RCC->APB1ENR |= RCC_APB1ENR_TIM3EN;
    RCC->AHBENR |= RCC_AHBENR_GPIOBEN;

    // Use alternate function for PB4
    MODIFY_FIELD(GPIOB->MODER, GPIO_MODER_MODER4, ESF_GPIO_MODER_ALT_FUNC);
    // Connect PB4 to TIM3_CH1
    MODIFY_FIELD(GPIOB->AFR[0], GPIO_AFRL_AFSEL4, 1);

    // Configure Time-Base
    TIM3->ARR = 0xffff;
    TIM3->SMCR = 0;
    TIM3->CR1 = 0; // count up
    TIM3->CR2 = 0;
    TIM3->EGR = 0;
    TIM3->PSC = TIM_PRESCALER - 1;

    // Select input TI1, enable CC1, rising edge detection
    TIM3->CCMR1 |= TIM_CCMR1_CC1S_0;
    TIM3->CCER |= TIM_CCER_CC1E;
    // Enable interrupts for capture and overflow/update
    TIM3->DIER |= TIM_DIER_CC1IE | TIM_DIER_UIE;
    TIM3->SR = 0;
}
```

```

NVIC_SetPriority(TIM3_IRQn, 128);
NVIC_ClearPendingIRQ(TIM3_IRQn);
NVIC_EnableIRQ(TIM3_IRQn);

// Enable TIM3
TIM3->CR1 |= TIM_CR1_CEN;
}

```

Listing 7.10 Function to initialize TIM3 with channel 1 in input capture mode.

First, we need to set up the TIM3 core to count. We will set the prescaler to divide by 4096, so the counting frequency is $48\text{ MHz}/4800 = 10\text{ kHz}$. Setting TIMx_ARR to 0xFFFF will set the overflow frequency to $10\text{ kHz}/65536 = 0.153\text{ Hz}$, or about once per 6.55 seconds.

Second, we will use channel 1 in input capture mode, so rising input edges will trigger the capture and also generate an interrupt request. The ISR will read the captured value from TIM3_CCR1 and compute the difference from the previous captured value. The difference is saved in a global variable `g_anem_period` for later conversion to wind speed by other (non-interrupt) code. The ISR will also set a flag indicating that there is new data available. Finally, if the time delay between two edges is too long, then the period will be set to zero to indicate invalid data.

The timer may overflow once or more during an anemometer rotation, so we can't simply find the difference between the current and previous timer values. For correct operation, we will extend the time range beyond 16 bits in software. We configure the TIM to generate an interrupt on timer overflow. The ISR will increment an overflow counter for each overflow that occurs. Because TIMx_ARR is set to 0xFFFF, each overflow count represents 0x10000 (65536) timer counts. As a result, we can create a 32-bit timestamp using the overflow count as the upper 16 bits and the counter value as the lower 16 bits. Finding the time difference involves taking the difference between two successive 32-bit timestamps.

Shown in Listing 7.11, the ISR sets and clears PA9 as a debug output signal to help visualize execution. The handler code must then determine why the ISR is running, as there are multiple possible trigger events: timer overflow (update) and input rising edge capture. The code handles the cases (including their combinations) and then clears the flags in TIM3->SR by writing ones to them. Note that this code will fail in a capture overwrite situation, when the ISR does not read the captured counter value before the next value is captured. For further details, please see the reference manual's sample code in Appendix A.9.4 (input capture data management code example).

```

void TIM3_IRQHandler(void) {
    static volatile uint32_t overflows = 0;
    static volatile uint32_t timer_val, prev_timer_val = 0;
    static uint32_t overflows_since_last_edge = 0;
    int32_t new_edge_capture = 0, new_overflow = 0;
    uint32_t extra_overflows = 0;

    // Debug signal : Entering ISR
    GPIOA->BSRR = GPIO_BSRR_BS_9;

    if (TIM3->SR & TIM_SR_CC1IF) { // Rising edge
        TIM3->SR = ~TIM_SR_CC1IF;
        timer_val = TIM3->CCR1;
        new_edge_capture = 1;
    }
}

```

```

if (TIM3->SR & TIM_SR_UIF) { // Update/Timer overflow
    TIM3->SR = ~TIM_SR_UIF;
    overflows++;
    new_overflow = 1;
    overflows_since_last_edge++;
    if ((!new_edge_capture) && (overflows_since_last_edge > MIN_SPEED_TIMEOUT)) {
        g_new_data = 1;
        g_anem_period = 0;
    }
}
if (new_edge_capture) {
    if (new_overflow && (TIM3->CCR1 > 0x8000)) {
        // overflowed after capture, so remove for this measurement
        extra_overflows = 1;
    }
    // calculate period
    timer_val |= (overflows - extra_overflows) << 16;
    g_anem_period = timer_val - prev_timer_val;
    prev_timer_val = timer_val;
    g_new_data = 1;
    overflows_since_last_edge = 0;
}
TIM3->SR = ~0; // clear all interrupt flags
// Debug signal: exiting ISR
GPIOA->BSRR = GPIO_BSRR_BR_9;
}

```

Listing 7.11 ISR to capture rising input edge and compute period since previous rising edge.

We will calculate the wind speed in the main thread rather than the ISR because the calculation involves executing comparatively slow code to perform floating-point math. The ISR passes the period information through the global variable `g_anem_period`, and also sets the `g_new_data` flag to indicate new data is available. Shown in Listing 7.14, the `main` function checks to see if `g_new_data` is true. If so, it will call the function in Listing 7.12 to calculate the wind speed. In this case, we want the wind speed in knots ($1 \text{ kt} = 1.852 \text{ km/h} = 0.514 \text{ m/s}$). Another function (Listing 7.13) is used to light the LEDs to indicate the wind speed.

```

#define M_PI (3.14159)
#define ANEM_R_MM (70)
#define ANEM_CLK_FREQ (F_TIM_CLOCK/TIM_PRESCALER)
#define KTS_PER_MM_S (0.00194384)

float Calculate_Windspeed_kt(uint32_t period) {
    float v;
    if (period > 0)
        v = KTS_PER_MM_S * 2 * M_PI * ANEM_R_MM * ANEM_CLK_FREQ / g_anem_period;
    else
        v = 0;
    return v;
}

```

Listing 7.12 Code to calculate wind speed.

```

void RGB_Windspeed(float v) {
    int r, g, b;
    r = g = b = 0;
    if (v < 0.1) // off
        r = 0;
    else if (v < 1) // red
        r = 1;
    else if (v < 3) // yellow
        r = g = 1;
    else if (v < 5) // green
        g = 1;
    else // white
        r = g = b = 1;
    Control_RGB_LEDS(r, g, b);
}

```

Listing 7.13 Code to display wind speed on RGB LED

```

int main(void) {
    uint32_t period;

    Set_Clocks_To_48MHz();
    Init_PA9_Debug();
    Init_TIM_IC();
    Init_GPIO_RGB();
    Control_RGB_LEDS(0,0,0);

    while (1) {
        if (g_new_data) {
            period = g_anem_period;
            g_new_data = 0;
            v_w = Calculate_Windspeed_kt(period);
            RGB_Windspeed(v_w);
        }
    }
}

```

Listing 7.14 `main` function initializes peripherals and then displays wind speed on RGB LEDs.

Output Modes

Each TIM channel has logic circuits that can detect when the TIM's counter CNT reaches a certain value. In response, the channel can change its output signal, trigger an interrupt, or trigger a DMA transfer (discussed in Chapter 9). The signal can change value when TIMx_CNT matches TIMx_CCRy. Various output signals are possible, including a single pulse, or a continuous stream of pulses with a specified duty cycle. The channel's flag CCyIF in TIMx_SR will be set to one when TIMx_CNT matches TIMx_CCRy. If CCyIE in TIMx_DIER is set, then TIMx will generate an interrupt.

Output Compare Mode

The most basic output mode is output compare mode. In this case, the channel will respond each time that `TIMx_CNT` reaches `TIMx_CCRy` (at the beginning of the cycle). The output can be configured to be set to one or cleared to zero on each match. This can be used to create a pulse with a fixed duration. After the first match, the output stays in its new state until the software explicitly changes it. The output can also be configured to toggle on each match, in which case the output signal is a square wave with a time offset (phase delay) determined by `TIMx_CCRy`.

PWM Signal Generation

Each channel of the TIM can generate a PWM signal with a frequency determined by `TIMx_ARR` and a duty cycle determined by `TIMx_CCRy`. The output signal's polarity can be selected to be active high or active low. Finally, the channel's pulses can be aligned by starting edges or centers.

Figure 7.24 shows an example of edge-aligned PWM mode, with `TIMx_ARR = 7` and `TIMx_CCRy = 2`. The TIMx counter `TIMx_CNT` counts up from zero to `TIMx_ARR`, so its count period is $(\text{TIMx_ARR} + 1)/f_{\text{count}}$.

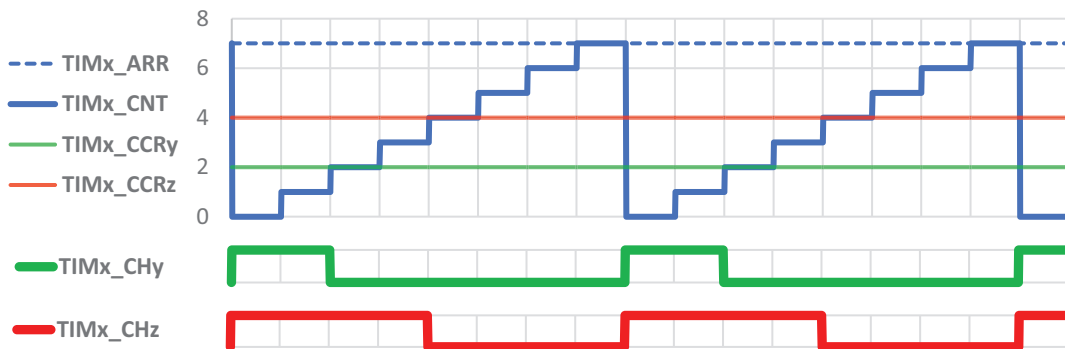


Figure 7.24 TIM operation with edge-aligned PWM.

Each channel's output is initialized (e.g. to one) each time counter `TIMx_CNT` overflows (e.g. from 7 to zero). `TIMx_CNT` counts and eventually matches `TIMx_CCRy`, at which point the channel's output signal `TIMx_CHy` is changed (e.g. to zero). `TIMx_CNT` continues counting up and eventually matches `TIMx_ARR` and overflows. At this point the cycle repeats. The resulting signal's pulse width is proportional to `TIMx_CCRy`, and the duty cycle is `TIMx_CCRy` divided by `TIMx_ARR`. Note that the channel's output's starting edge is aligned with the overflow of `TIMx_CNT`. If additional channels are enabled, they will all have starting edges at the same time.

Figure 7.25 shows an example of center-aligned PWM mode. The TIM counter `TIMx_CNT` alternates between counting down and up, so its count period is $2 * \text{TIMx_ARR} / f_{\text{count}}$.

`TIMx_CNT` is initialized to `TIMx_ARR`, which also initialized the channel output (e.g. to zero). `TIMx_CNT` then counts down and eventually matches `TIMx_CCRy`, at which time the channel output is toggled (e.g. zero to one). `TIMx_CNT` keeps counting and eventually reaches zero, at which point it changes count direction and starts counting up. `TIMx_CNT` eventually matches `TIMx_CCRy`, at which time the channel output is toggled (e.g. one to zero).

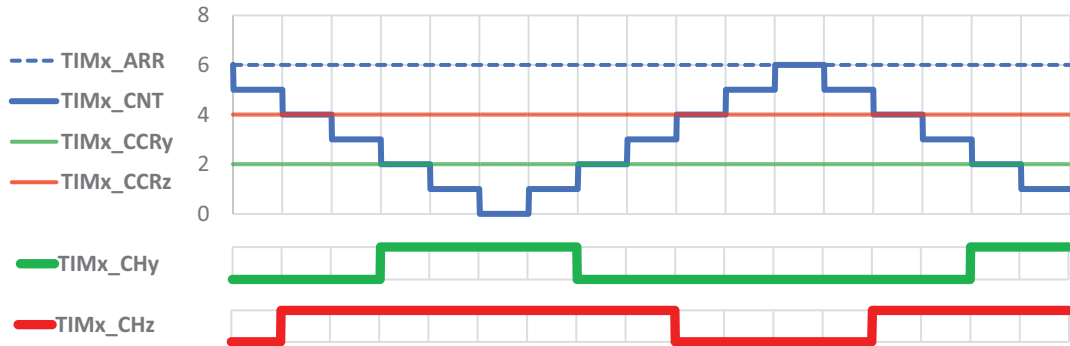


Figure 7.25 TIM operation with center-aligned PWM.

TIMx_CNT continues until it reaches TIMx_ARR, at which point it changes count direction and starts counting down. At this point the cycle repeats. The resulting signal's pulse width is proportional to TIMx_CCRy, and the duty cycle is $(2 * \text{TIMx_CCRx}) / (2 * \text{TIMx_ARR}) = \text{TIMx_CCRx} / \text{TIMx_ARR}$.

Note that each channel's output's signal is centered on the transition of TIMx_CNT from one to zero. This center alignment is useful for switching circuits that require a time delay (dead time) between deactivating some components and enabling others (e.g. a synchronous buck power converter).

Example: LED Dimming

Let's drive an LED with a PWM output so it gradually brightens and dims the LED by increasing or decreasing the duty cycle of the LED drive signal. The external RGB LED is connected to the Nucleo-F091RC as in Chapter 2, with the blue LED connected to pin PA7. Please refer to Figure 7.8 for signal locations. We examine the MCU datasheet to find the alternate functions assignments for PA7 [6]. Pin PA7 can be connected to TIM3 channel 2 (TIM3_CH2) by setting its alternate function to AF1. So we will need to use TIM3 channel 2 to drive the LED.

Listing 7.15 shows the code to initialize the TIM to drive the blue LED with a PWM signal. We will use edge-aligned up-counting mode for simplicity. Because the LED will be lit when the output is low, we will configure the channel to generate an active low output so that setting TIMx_CCRy to zero will turn off the LED. To turn the LED fully on we set TIMx_CCRy to TIMx_ARR.

The frequency of the generated PWM signal should be at least 50 Hz to prevent visible flickering. Let's pick 500 Hz as the PWM frequency. The TIM input clock frequency of 48 MHz will need to be divided down to 500 Hz, so we need a division factor of 96000. We will use the prescaler to get this value down to no more than 65536, which is the maximum division factor for our 16-bit TIMx_ARR register. We will need a prescaler factor of at least $96000 / 65536 = 1.46$, so any factor of 2 or more is sufficient. Using the prescaler factor of 2 means the TIMx_ARR value should be $96000 / 2 - 1 = 48000 - 1 = 47999$. We will be able to set the LED to one of 48000 brightness levels.

Note that the larger the prescaler factor, the fewer different PWM values will be available. Using the prescaler factor of 128 means the TIMx_ARR value should be $96000 / 128 - 1 = 750 - 1 = 749$. We would then be able to set the LED to one of only 750 brightness levels. For some applications this reduced resolution might be a problem.

For this example we do not enable interrupts. However, if we did, they would occur at 500 Hz, which could serve as a useful timing reference for other parts of the program.

```
#define F_TIM_CLOCK (48UL*1000UL*1000UL) // 48 MHz
#define PWM_FREQUENCY (500)
#define PWM_MAX_DUTY_VALUE ( (F_TIM_CLOCK / (PWM_FREQUENCY * PWM_PRESCALER)) - 1)
#define PWM_PRESCALER (2)

void Init_Blue_LED_PWM(void) {
    // Configure PA7 (blue LED) with alternate function 1: TIM3_CH2
    RCC->AHBENR |= RCC_AHBENR_GPIOAEN;
    MODIFY_FIELD(GPIOA->MODER, GPIO_MODER_MODER7, ESF_GPIO_MODER_ALT_FUNC);
    MODIFY_FIELD(GPIOA->AFR[0], GPIO_AFRL_AFRL7, 1);

    // Configure TIM3 counter and prescaler
    RCC->APB1ENR |= RCC_APB1ENR_TIM3EN;
    TIM3->PSC = PWM_PRESCALER - 1;
    TIM3->ARR = PWM_MAX_DUTY_VALUE;
    TIM3->CR1 = 0; // count up

    // Configure TIM3 channel 2
    TIM3->CCR2 = 1; // Short on-time by default
    TIM3->CCER |= TIM_CCER_CC2P; // active low polarity
    MODIFY_FIELD(TIM3->CCMR1, TIM_CCMR1_OC2M, 6); // Select PWM mode
    TIM3->CCMR1 |= TIM_CCMR1_OC2PE; // Enable preload register
    TIM3->EGR |= TIM_EGR_UG; // Generate update
    TIM3->CCER |= TIM_CCER_CC2E; // Enable channel output on OC2
    TIM3->BDTR |= TIM_BDTR_MOE; // Enable main output

    TIM3->CR1 |= TIM_CR1_CEN; // Enable timer
}
```

Listing 7.15 Function to initialize TIM3 Channel 2 to drive blue LED.

Listing 7.16 shows the `main` function that first initializes the timer and then adjusts the LED brightness. An infinite loop first sets the LED brightness by writing the pulse width to CCR2. Then it adjusts the `pulse_width` variable to ramp up and down between 0 and `PWM_MAX_DUTY_VALUE`. Finally, we use a software delay loop that counts up to `DIM_DELAY`. This delay makes the brighten/dim cycle slow enough to see. It would be simple to modify this program to use the timer's ISR to adjust the signal's duty cycle, freeing up most of the processor's time for other processing.

```
#define DIM_DELAY (100)

int main(void) {
    int pulse_width = 0, change = 1;
    volatile int32_t delay;

    Set_Clocks_To_48MHz();
    Init_Blue_LED_PWM();

    // Flash forever
    while (1) {
        TIM3->CCR2 = pulse_width;
        pulse_width += change;
```

```

    if (pulse_width <= 0)
        change = 1;
    else if (pulse_width >= PWM_MAX_DUTY_VALUE)
        change -= 1;
    // Delay before changing pulse width again
    for (delay = 0; delay < DIM_DELAY; delay++)
        ;
}
}

```

Listing 7.16 Code to gradually brighten and dim the blue LED.

Summary

This chapter has introduced three types of timer peripherals, the Cortex-M SysTick timer, the STM32F091RC WWDG watchdog timer, and the STM32F091RC TIM. The SysTick timer has a 24-bit counter and can be used to serve as a time reference or generate periodic interrupts or a time delay. The WWDG watchdog timer enables a system to automatically detect and recover from faults or bugs that keep the watchdog from being serviced. The TIM has a 16-bit or 32-bit counter and channels. The TIM core has similar capabilities to the SysTick timer, but is enhanced by multiple channels that can be used in input capture or output generation mode. Input capture mode performs precise timing measurements on digital input signals. Output generation mode generates digital output signals (e.g. with pulse-width modulation) with precise timing.

Exercises

1. Specify how the SysTick control registers must be configured so that the timer generates interrupts with a frequency of 315 Hz, assuming a clock of 48 MHz. What is the actual frequency generated?
2. If the SysTick timer generates interrupts at 19199 Hz and the bus clock frequency is 48 MHz, what values are in the RELOAD and CLKSOURCE fields?
3. If the SysTick Load register contains 0x00394391 and the CLKSOURCE is one, what is the interrupt period?
4. What is the lowest interrupt frequency that the SysTick timer on a STM32F091RC MCU can generate? Assume the CPU clock is 48 MHz.
5. What is the shortest time-out period available with the WWDG? Show the register settings needed.
6. What is the longest time-out period available with the WWDG? Show the register settings needed.
7. Specify how the control registers must be configured so that TIM2 generates interrupts at an approximate frequency of 2017.0101 Hz. What is the actual frequency of interrupts?
8. Specify how the control registers must be configured so that TIM3 channel 3 generates pulses that are high for 150 μ s and low for 27 μ s, assuming an input clock of 24 MHz. No interrupts are to be generated, but the pulses are to be generated on MCU Port B bit 0. What are the actual high and low times?

9. Specify how the control registers must be configured so that one of the channels in TIM3 measures the delay until the pulse applied to Port A bit 6 changes from one to zero. TIM also must generate an interrupt at that time. Each count in TIMx_CCRy must represent one-third of a microsecond.
10. What is the lowest interrupt frequency that a 16-bit Timer/PWM module on a STM32F091RC MCU can generate? Assume the bus clock is 48 MHz.

References

- [1] Arm Ltd., *ARMv6-M Architecture Reference Manual*, DDI 0419D, 2017.
- [2] J. Yiu, *The Definitive Guide to the ARM Cortex-M0 and Cortex-M0+ Processors*, 2nd ed., Oxford: Newnes, 2015.
- [3] STMicroelectronics NV, *Reference Manual RM0091: STM32F0x1/STM32F0x2/STM32F0x8*, 2017.
- [4] P. Koopman, *Better Embedded System Software*, Pittsburgh: Drumnadrochit Education LLC, 2010.
- [5] J. Ganssle, *The Art of Designing Embedded Systems*, Second ed., Elsevier Inc., 2008.
- [6] STMicroelectronics NV, *STM32F091xB STM32F091xC Data Sheet*, DocID 026284, 2017.

The `Init_SysTick` function in Listing 7.1 configures the timer. It first writes the RELOAD value, enables the SysTick IRQ in the NVIC, initializes the current count value, and finally configures CTRL to use the alternate clock source, enable interrupts, and enable the timer.

The exception handler for the SysTick Timer in Listing 7.2 runs each time the timer reaches zero, which is 1 Hz in this example. The handler lights the R, G and B LEDs based on the values of bits 0, 1, and 2 in variable `n`.

```
void SysTick_Handler(void) {
    static int n = 0;
    Control_RGB_LEDS(n & 1, n & 2, n & 4);
    n++;
}
```

Listing 7.2 Handler for SysTick exception runs every second, changing LED color.

Listing 7.3 shows the main function, which initializes the clocks and peripherals and then enters an infinite loop, letting `SysTick_Handler` do its work as needed.

```
int main(void) {
    Set_Clocks_To_48MHz();
    Init_GPIO_RGB();
    Init_SysTick();
    SystemCoreClockUpdate();
    while (1)
        ;
}
```

Listing 7.3 `main` function for SysTick example.

STM32F091RC Watchdog Timers

The STM32F091RC MCU features two watchdog timers called the Independent Watchdog (IWDG) and the System Window Watchdog (WWDG). These watchdogs will reset the MCU if it is not serviced when required. The IWDG is driven by its own dedicated clock and can stay active even if the main clock fails. This watchdog is not discussed further as this chapter focuses on the WWDG. The MCU documentation holds more information about both the WWDG and IWDG [3].

Figure 7.9 shows the basic operation of the WWDG. When the MCU comes out of reset, the program starts running and can configure the WWDG operation. The WWDG counts down as the application program runs. Eventually the program services the WWDG, resetting its counter to its initial value. The WWDG resumes counting from that initial value, expecting to be serviced again by the program before reaching its time-out value. The diagram shows a case in which the program starts running out of control (e.g. due to a bug or electrical noise) and the WWDG is not serviced. Eventually, the WWDG times out, resetting the MCU and causing the program to restart.

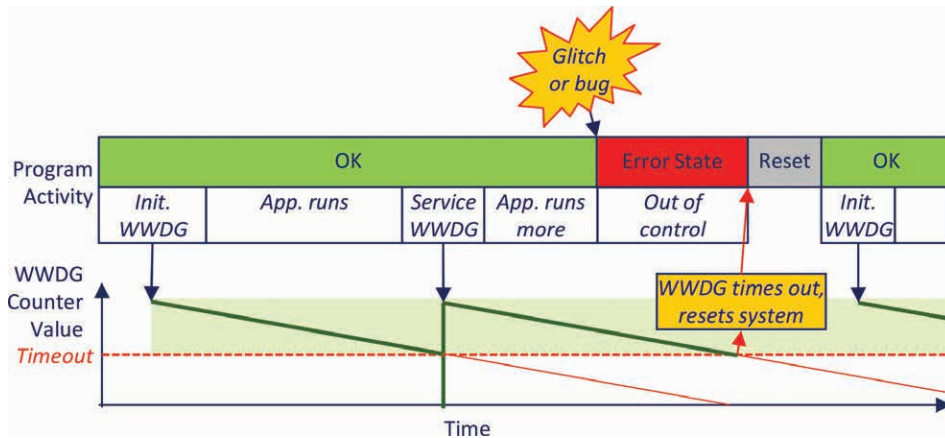


Figure 7.9 The WWDG watchdog timer detects program that is out of control, resets system to restart program.

With the window mode, the WWDG must be serviced within a specific portion of the time period, as shown in Figure 7.10 in green. This provides more robust protection against faults, because if the watchdog is serviced too early or too late (outside of the green window area) it will reset the CPU.

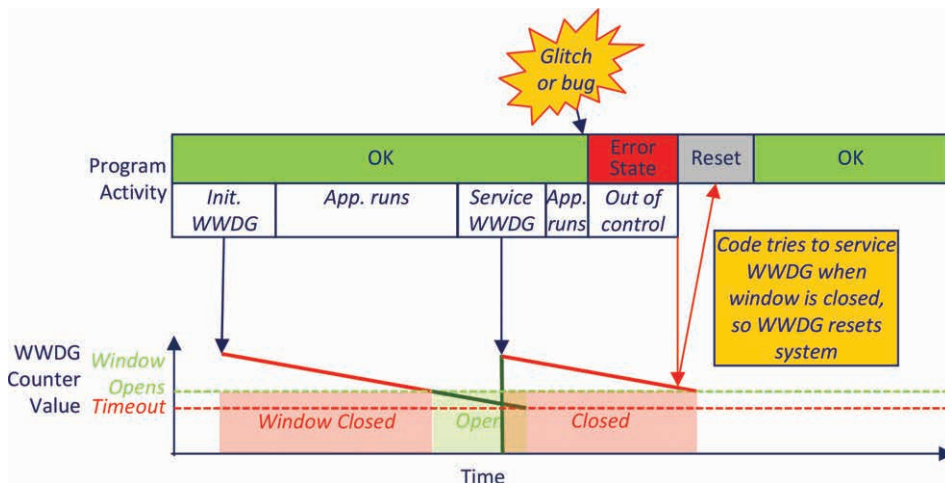


Figure 7.10 Windowed watchdog resets system if serviced outside of valid time window.

Hardware Configuration

The peripheral clock for the WWDG must be enabled by setting the bit `WWDGEN` in `RCC_APB1ENR`.

Figure 7.11 shows the WWDG's control register (`WWDG_CR`).

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WDGA	T						
Reset									0	1	1	1	1	1	1	1
Access									r*	rw	rw	rw	rw	rw	rw	rw

Figure 7.11 System window watchdog control register (WWDG_CR).

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	Res.	Res.	Res.	Res.	Res.	Res.	EWI	WDGTB		W						
Reset							0	0	0	1	1	1	1	1	1	1
Access							r*	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 7.12 System window watchdog configuration register (WWDG_CFR).

- The watchdog is activated by setting the WDGA field of WWDG_CR by software. This field can only be reset by hardware after a reset occurred.
- The window for the watchdog is selected in the Configuration Register (WWDG_CFR) by writing the 7-bit field W. By default its value is 0x7F which means that the watchdog can be serviced at any time.
- The WDGTB field of WWDG_CFR selects the prescaler. Various prescaler values are available: the bus clock divided by 4096 (00), 2*4096 (01), 4*4096 (10) or 8*4096 (11).
- The T field contains the 7-bit counter of the watchdog. A reset is generated when this counter is decremented from 0x40 to 0x3F (in other words when the bit 6 of the counter is switched from 1 to 0).

The amount of time until the WWDG times out depends on the APB clock period, the WDG prescaler, and the value of the T field. The timeout value is calculated as:

$$t_{\text{WWDG}} = t_{\text{PCLK}} * 4096 * 2^{\text{WDGTB}[1:0]} * (T[5:0] + 1)$$

With a 48 MHz bus clock, WDGTB = 3 and T[5:0] = 63, the WWDG will time out after 43.69 ms.

To service the WWDG, the software needs to reload the 7-bit counter by writing to the T field of WDG_CR. If the counter is reloaded when its value is higher than the window value in the W field of WDG_CFR then a reset is generated.

The processor can read the Control and Status Register of the RCC module (RCC_CSR) to determine the cause of the reset. The WWDGRSTF bit is set to one if a reset has been caused by the system window watchdog timer. Other causes which can be distinguished include independent watchdog reset, low-power reset and external reset pin. These flags are cleared by software by writing to the RMVF field in RCC_CSR.

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Field	LPWR RSTF	WWDG RSTF	IWDG RSTF	SFT RSTF	POR RSTF	PIN RSTF	OB RSTF	RMVF	V18PW R RSTF	Res.	Res.	Res.	Res.	Res.	Res.	Res.
Reset	x	x	x	x	x	x	x	x	x							
Access	r	r	r	r	r	r	r	rw	r							

Figure 7.13 Control and Status register of the RCC module (RCC_CSR) indicates cause of reset. Only the bits 31 to 16 are shown here.

How to Use a WDT

Now we know how to configure and service the WWDG. How should we use it? Here are some common best practices; these and others are discussed in greater depth elsewhere [4] [5].

How Long Should the WDT Period Be?

The WDT period sets the maximum detection delay, so choose one that is appropriate for the system. An MCU controlling a cordless power tool (e.g. a drill) needs a short WDT period because the tool can quickly injure the user, or damage the environment, the drill itself, and the battery.

If we shorten its period, the WDT must be serviced more frequently. This requires a better understanding of the program's timing behavior, which may be more complex than expected.

Where Should the WDT Be Serviced, and How Often?

Do not service the watchdog in an interrupt service routine, because ISRs are likely to run even if the rest of the software has crashed. Instead service the watchdog in mainline non-ISR code, preferably low-priority code that is vulnerable to system crashes.

Do not scatter WDT service commands throughout the code. First, this lack of design leads to sloppy use of the WDT, making it much less effective. Second, having multiple WDT service locations will complicate finding the cause of WDT resets.

Embedded systems have code for two phases: the start-up code configures and prepares the system, whereas the operational code handles everything else. The WDT may need to be serviced multiple times during system startup if there are operations that take a long time to complete. Once the system is in its operational mode, the WDT should be serviced in few places, preferably only one.

How Do We Debug a System with a WDT?

A WDT complicates debugging, as it will reset the system shortly after the program hits a breakpoint. To prevent this problem, the STM32F091RC WWDG will not run while the MCU is in debug or stop mode if the DBG_WWDG_STOP field in the Debug MCU APB1 Freeze Register (DBGMCU_APB1_FZ) is set. Other WDTs typically have an equivalent feature.

Example: Long Error Condition

We can use the WWDG to reset the program when user switch B1 is pressed down for too long.

After starting up, the code will initialize the LEDs and switches. It will then flash the RGB LED five times to indicate if this is a start-up after a regular reset (green) or after a WDT reset (red). The code will next initialize the WWDG. Finally the code will enter its main loop, in which it

reads switch B1. If the switch is not pressed, the LED will be lit blue and the WWDG will be serviced. If the switch is pressed, the LED will be turned off and the WWDG will not be serviced. The main loop then repeats. If the switch is held down for too long, the WWDG will not be serviced before it times out, resetting the system. The code will restart, flashing the LED red to indicate the WDT reset cause.

The two functions to initialize and service the WWDG are shown in Listing 7.4 and are quite simple.

```
void Init_WWDG(void) {
    // Enable peripheral clock for WWDG
    RCC->APB1ENR |= RCC_APB1ENR_WWDGEN;
    // Select the bus clock divided by 4096 * 8
    MODIFY_FIELD(WWDG->CFR, WWDG_CFR_WDGTB, 3);
    // Set window to be fully open - refresh anytime
    MODIFY_FIELD(WWDG->CFR, WWDG_CFR_W, 0x7f);
    // Load counter with initial counter value
    MODIFY_FIELD(WWDG->CR, WWDG_CR_T, 0x7f);
    // Activate the WWDG
    WWDG->CR |= WWDG_CR_WDGA;
}

void Service_WWDG(void) {
    MODIFY_FIELD(WWDG->CR, WWDG_CR_T, 0x7f);
}
```

Listing 7.4 Functions to initialize and service the WWDG.

The code to determine the cause of the reset and flash the LEDs accordingly is shown in Listing 7.5. The code reads the control and status register of the RCC module and lights the correct LEDs. After a delay, all the LEDs are turned off. After another delay the loop repeats or exits. At the end of the function the RCC_CSR register reset flags are reset.

```
#define NUM_STARTUP_FLASHES (5)
#define STARTUP_FLASH_DURATION (50)

void Flash_Reset_Cause() {
    unsigned int n;
    for (n = 0; n < NUM_STARTUP_FLASHES; n++) {
        if (RCC->CSR & RCC_CSR_WWDGRSTF) {
            // Red: WWDG caused reset
            Control_RGB_LEDS(1, 0, 0);
        } else {
            // Green: WWDG did not cause reset
            Control_RGB_LEDS(0, 1, 0);
        }
        Delay(STARTUP_FLASH_DURATION);
        Control_RGB_LEDS(0, 0, 0);
        Delay(10 * STARTUP_FLASH_DURATION);
    }
    // Clear all reset cause flags
    RCC->CSR |= RCC_CSR_RMVF;
}
```

Listing 7.5 Function to flash LEDs with color determined by the reset cause.

```

int main(void) {
    Set_Clocks_To_48MHz();
    Init_GPIO_RGB();
    Init_GPIO_Switches();
    // Show why system is starting up by flashing LEDs
    Flash_Reset_Cause();
    // Start up watchdog
    Init_WWDG();
    // Main loop: do or don't refresh watchdog
    while (1) {
        if (SWITCH_PRESSED(SW1_POS)) {
            // Don't service watchdog
            // Turn off LEDs - warning!
            Control_RGB_LEDs(0, 0, 0);
        } else {
            // Service watchdog
            Service_WWDG();
            // Light blue LED - OK!
            Control_RGB_LEDs(0, 0, 1);
        }
    }
}

```

Listing 7.6 main function for using WWDG to reset system when switch B1 is pressed for too long.

The `main` function is shown in Listing 7.6. If switch B1 is pressed for too long, the watchdog will expire and reset the system. Given that the $t_{\text{WWDG}} = 43.69 \text{ ms}$, the switch must be pressed and released very quickly. After some practice, you should be able to gently tap the switch and not trigger the watchdog.

One can monitor the switch and LED signals; please refer to Figure 7.8 for connection locations. Figure 7.14 shows an example of a long switch press which resets the system, while Figure 7.15 shows a short press which does not.

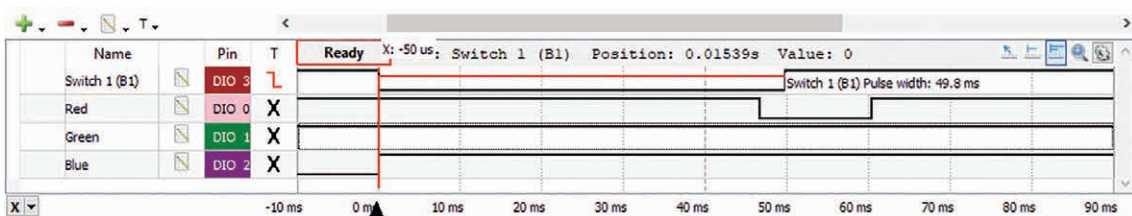


Figure 7.14 Switch is pressed for too long (49.8 ms), causing WWDG to reset the system. Note red LED is lit starting at about 47 ms to indicate reset due to watchdog.

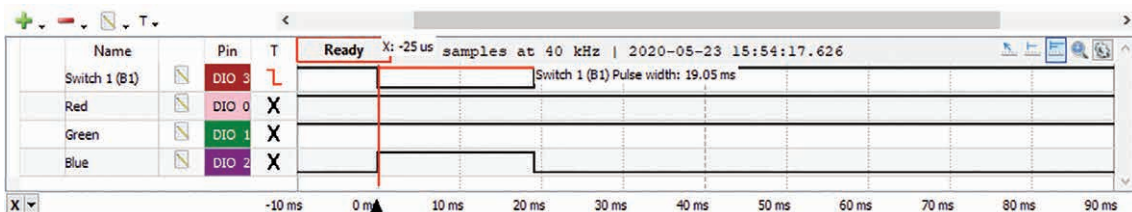


Figure 7.15 Switch is pressed for about 19 ms, so WWDG does not reset system.

STM32F091RC Timer and PWM Module

The timer module (TIM) consists of a time-base unit (with a 16-bit or 32-bit counter) and multiple channels that use the core counter's value to measure the timing of input signals or generate output signals. Figure 7.16 shows a block diagram of the TIM circuitry. To use a TIM peripheral, enable its peripheral clock in the RCC. For example, enable TIM3's peripheral clock by setting the field TIM3EN in RCC->APB1ENR to 1.

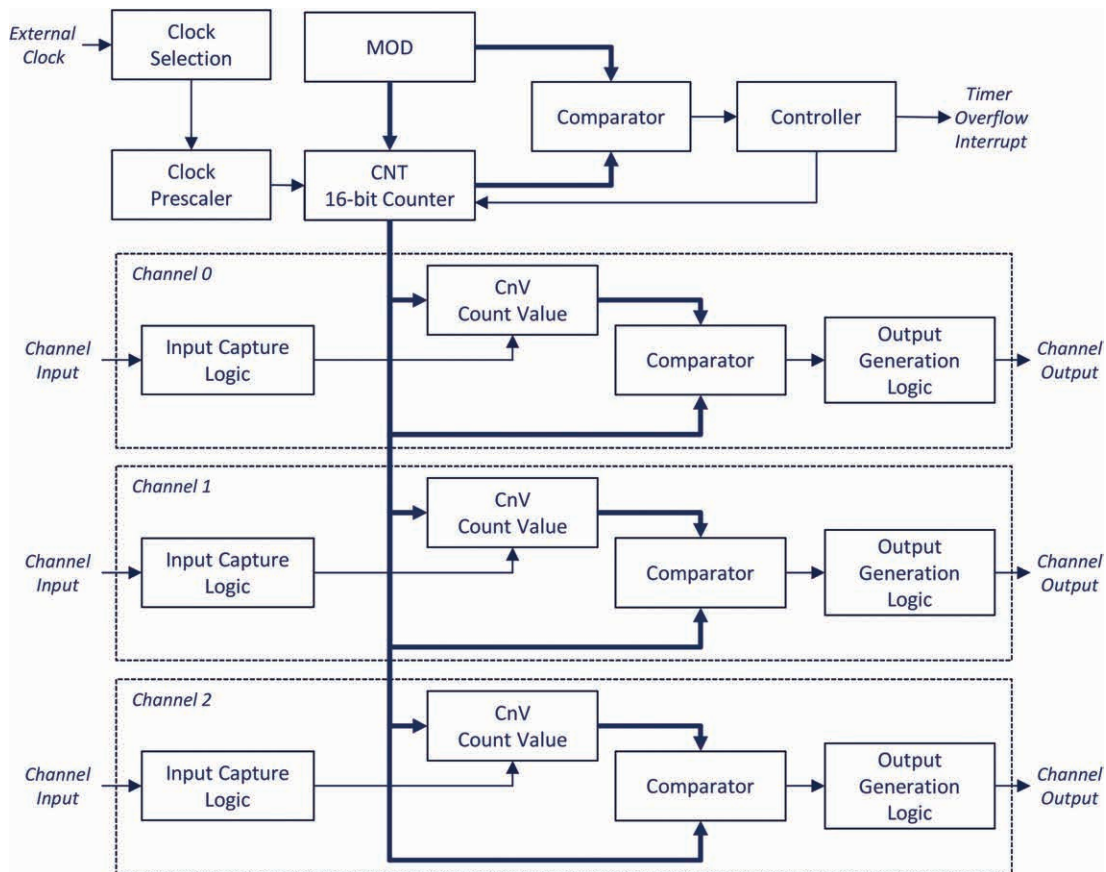


Figure 7.16 Timer peripheral with time-base unit and multiple channels for input and output processing.

The STM32F091RC MCU has 10 TIMs: one 16-bit advanced-controller timer (TIM1) with four channels, one 32-bit timer (TIM2) with four channels and seven 16-bit timers with up to four channels (TIM3, TIM6, TIM7, TIM14, TIM15, TIM16 and TIM17). Full information is available in the documentation [3] [6].

This chapter refers specifically to TIM3. Other timers available on STM32F091RC may have more or fewer features and some of the registers may differ slightly. You can refer to the reference manual for more information [3].